

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO3: Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)

Assessment Tool Weightage: 40%

QUESTIONS:5

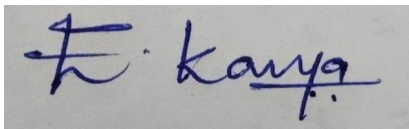
Duration :60 Mins
On Class
Total Marks:50

Annexure A

EXPERIMENTS

Code of Conduct:

I E.Kavya(192424365) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Annexure A

EXPERIMENTS

Q1. Write a C program to implement a syntax-directed definition for evaluating arithmetic expressions using the following grammar:

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow \text{num}$$

Input:

3 + 5 * 2

The program should display the attribute values and the final value of the expression.
(10 Marks)

Q2. Write a C program to implement a simple type checker for arithmetic expressions using the following grammar:

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow \text{id}$$

Consider the following symbol table:

a : int

b : int

c : float

Input:

a + b * c

Determine the type of each subexpression and the final expression. (10 Marks)

Q3. Write a C program to determine whether two type expressions are equivalent.

Consider the following type expressions:

T1 = array(10, int)

T2 = array(10, int)

The program should display:

Equivalent or Not Equivalent . (10 Marks)

Q4. Write a C program to construct and display the syntax tree for the following expression using the given grammar:

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow \text{id}$

Input:

$a + b * c$

Display the constructed syntax tree. (10 Marks)

Q5. Write a C program to demonstrate the propagation of inherited and synthesized attributes using the following grammar:

$D \rightarrow T L$

$T \rightarrow \text{int}$

$L \rightarrow \text{id } L'$

$L' \rightarrow , \text{id } L' \mid \epsilon$

Input:

int a,b,c

Display the type assigned to each identifier. (10 Marks)

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

C03-AT1

EXPERIMENTS

Q1. Syntax-Directed Definition for Arithmetic Expression

Aim

To write a C program to implement a syntax-directed definition for evaluating arithmetic expressions using the given grammar and display the attribute values and final value.

Grammar

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow \text{num}$$

procedure

1. Start the program.
2. Assign the input values 3, 5, and 2.
3. Apply $F \rightarrow \text{num}$ to obtain the attribute values.
4. Evaluate $5 * 2$ using $T \rightarrow T * F$.
5. Evaluate $3 + 10$ using $E \rightarrow E + T$.
6. Display the intermediate attribute values.
7. Display the final value.
8. Stop the program.

Program

```
#include <stdio.h>
int main()
{
    int a = 3;
    int b = 5;
    int c = 2;
    int t_value;
    int e_value;
    printf("F -> num = %d\n", a);
    printf("F -> num = %d\n", b);
    printf("F -> num = %d\n", c);
    t_value = b * c;
    printf("T -> T * F = %d * %d = %d\n", b, c, t_value);
    e_value = a + t_value;
    printf("E -> E + T = %d + %d = %d\n", a, t_value, e_value);
    printf("\nFinal value = %d\n", e_value);
}
```

```
    return 0;  
}
```

Input

3 + 5 * 2

Sample Output

```
Output  
  
F -> num = 3  
F -> num = 5  
F -> num = 2  
T -> T * F = 5 * 2 = 10  
E -> E + T = 3 + 10 = 13  
  
Final value = 13
```

Result

Thus, the arithmetic expression 3 + 5 * 2 was evaluated successfully using syntax-directed definition.

Q2. Simple Type Checker

Aim

To write a C program to implement a simple type checker for arithmetic expressions and determine the type of each subexpression and the final expression.

Grammar

$E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow \text{id}$

Symbol Table

Identifier Type

a	int
b	int
c	float

Procedure

1. Start the program.
2. Create the symbol table with types of a, b, and c.
3. Read the expression $a + b * c$.
4. Determine the type of $b * c$.
5. Since b is int and c is float, the multiplication result is float.
6. Determine the type of $a + (b * c)$.
7. Since one operand is float, the final result is float.
8. Display all subexpression types.
9. Stop the program.

Program

```
#include <stdio.h>
#include <string.h>
int main()
{
    char a_type[] = "int";
    char b_type[] = "int";
    char c_type[] = "float";
    char multiply_type[10];
    char final_type[10];
    printf("Symbol Table:\n");
    printf("a : %s\n", a_type);
    printf("b : %s\n", b_type);
    printf("c : %s\n\n", c_type);
    if (strcmp(b_type, "float") == 0 ||
        strcmp(c_type, "float") == 0)
    {
        strcpy(multiply_type, "float");
    }
    else
    {
        strcpy(multiply_type, "int");
    }
    printf("Type of b * c = %s\n", multiply_type);
    if (strcmp(a_type, "float") == 0 ||
        strcmp(multiply_type, "float") == 0)
    {
        strcpy(final_type, "float");
    }
    else
    {
        strcpy(final_type, "int");
    }

    printf("Type of a + b * c = %s\n", final_type);
    printf("\nFinal expression type = %s\n", final_type);
}
```

```
    return 0;  
}
```

Input

$a + b * c$

Sample Output

```
Output  
  
Symbol Table:  
a : int  
b : int  
c : float  
  
Type of b * c = float  
Type of a + b * c = float  
  
Final expression type = float
```

Result:

Thus, the type of $b * c$ is float and the final expression $a + b * c$ is also float.

Q3. Type Expression Equivalence

Aim

To write a C program to determine whether two given type expressions are equivalent.

Given

T1 = array(10, int)

T2 = array(10, int)

Procedure

1. Start the program.
2. Store the size and element type of T1.
3. Store the size and element type of T2.
4. Compare the array sizes.
5. Compare the element types.
6. If both are the same, display Equivalent.
7. Otherwise, display Not Equivalent.
8. Stop the program.

Program

```

#include <stdio.h>
#include <string.h>
int main()
{
    int size1 = 10;
    int size2 = 10;
    char type1[] = "int";
    char type2[] = "int";
    printf("T1 = array(%d, %s)\n", size1, type1);
    printf("T2 = array(%d, %s)\n\n", size2, type2);
    if (size1 == size2 && strcmp(type1, type2) == 0)
    {
        printf("Equivalent\n");
    }
    else
    {
        printf("Not Equivalent\n");
    }
    return 0;
}

```

Output

```

T1 = array(10, int)
T2 = array(10, int)

Equivalent

```

Result

Thus, the type expressions T1 and T2 are equivalent.

Q4. Syntax Tree Construction

Aim

To write a C program to construct and display the syntax tree for the expression $a + b * c$.

Grammar

$$\begin{aligned}
 E &\rightarrow E + T \mid T \\
 T &\rightarrow T * F \mid F \\
 F &\rightarrow \text{id}
 \end{aligned}$$

Procedure

1. Start the program.
2. Create a node structure containing data, left child, and right child.
3. Create + as the root node.
4. Create a as the left child of +.
5. Create * as the right child of +.
6. Create b and c as the left and right children of *.
7. Display the constructed syntax tree.
8. Stop the program.

Program

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
    char data;
    struct Node *left;
    struct Node *right;
};
struct Node* createNode(char data)
{
    struct Node *newNode;
    newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}
void displayTree(struct Node *root, int space)
{
    int i;
    if (root == NULL)
        return;
    space = space + 5;
    displayTree(root->right, space);

    printf("\n");

    for (i = 5; i < space; i++)
        printf(" ");
    printf("%c\n", root->data);
    displayTree(root->left, space);
}
int main()
{
    struct Node *root;
    root = createNode('+');
    root->left = createNode('a');
    root->right = createNode('*');
    root->right->left = createNode('b');
```

```

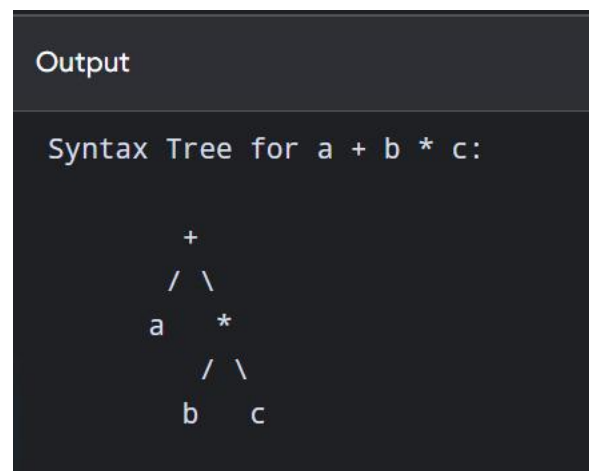
    root->right->right = createNode('c');
    printf("Syntax Tree for a + b * c:\n");
    displayTree(root, 0);
    return 0;
}

```

Input

a + b * c

Sample Output



Result

Thus, the syntax tree for a + b * c was successfully constructed and displayed.

Q5. Inherited and Synthesized Attributes

Aim

To write a C program to demonstrate the propagation of inherited and synthesized attributes and assign the type int to each identifier.

Grammar

$D \rightarrow T L$
 $T \rightarrow \text{int}$
 $L \rightarrow \text{id } L'$
 $L' \rightarrow , \text{id } L' \mid \epsilon$

Procedure

1. Start the program.
2. Apply $T \rightarrow \text{int}$.

3. Store int as the type attribute.
4. Identify the identifiers a, b, and c.
5. Propagate the int type from T to each identifier.
6. Display the type assigned to each identifier.
7. Stop the program.

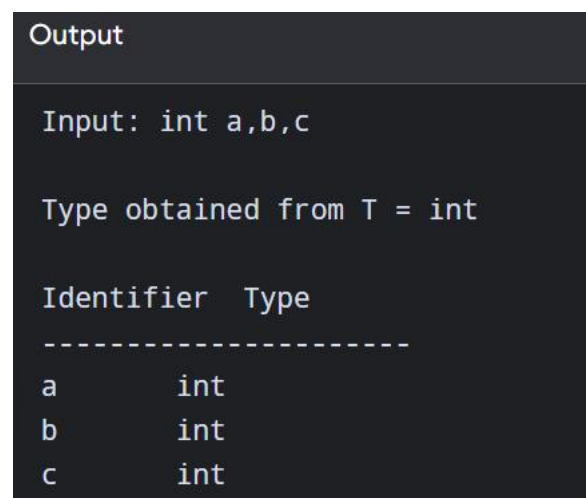
Program

```
#include <stdio.h>
int main()
{
    char type[] = "int";
    char id1[] = "a";
    char id2[] = "b";
    char id3[] = "c";
    printf("Input: int a,b,c\n\n");
    printf("Type obtained from T = %s\n\n", type);
    printf("Identifier\tType\n");
    printf("-----\n");
    printf("%s\t\t%s\n", id1, type);
    printf("%s\t\t%s\n", id2, type);
    printf("%s\t\t%s\n", id3, type);
    return 0;
}
```

Input

int a,b,c

Sample Output



```
Output

Input: int a,b,c

Type obtained from T = int

Identifier  Type
-----
a          int
b          int
c          int
```

Result

Thus, the inherited type int was successfully propagated to all identifiers a, b, and c.